

ADVANCED GENAI MODULE

Day 2: Making LLMs Powerful

From Passive Models to Active Problem Solvers

Course Roadmap: From Foundations to Capstone

01



Understanding the Fundamental

LLM architecture, history, and core prompt engineering.

02



Making Agents Powerful

RAG & Tool usage

03



Working with Frameworks

LangChain, LangGraph, and building complex chains.

04



Observability & Fine Tuning

Tracing (LangSmith), evaluation, and custom model tuning.

05



Capstone Project

End-to-end implementation and final presentation.

Day 2 Agenda



End to end RAG Implementation & Advanced RAG Patterns



Advanced RAG Patterns

Lunch Break (40 Minute)



Multimodal RAG



Tool Usage & Function Calling Giving agents the ability to perform actions

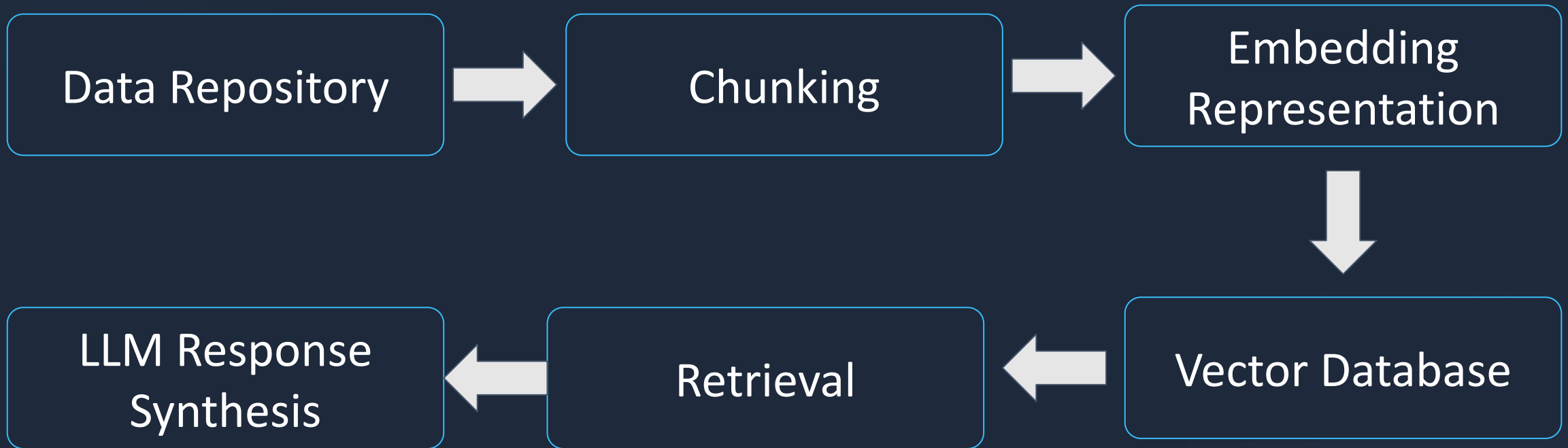
QnA

Section 1: Basic RAG & Advanced RAG Patterns

- ✓ RAG End to End Pipeline
- ✓ Query transformation
- ✓ Hybrid Search

✔ RAG End to End Pipeline

RAG End to End Pipeline



Data Repository



Many More....

- LangChain Document Loaders convert data from various formats (e.g., CSV, PDF, HTML) into standardized **Document objects** for LLM applications.
- **Langchain Document Loader:**
https://docs.langchain.com/oss/python/integrations/document_loaders

Chunking

It is the process of breaking large documents -

- into smaller,
- overlapping text segments

so they can be efficiently embedded, retrieved, and processed by language models while preserving context.

What do you mean by overlapping segments

Chunking

What do you mean by overlapping segments?

Chunking Strategies

- Fixed-size chunks (token or character count)
- Semantic chunking (paragraphs, sections)
- Recursive chunking with hierarchical representation

Chunk overlap: Each chunk shares a configurable number of tokens/characters with adjacent chunks, ensuring important context isn't lost at boundaries.

Recursive splitting: `RecursiveCharacterTextSplitter` respects natural text structure (paragraphs, sentences, sections) before falling back to smaller units, keeping related information together.

Examples

Document Text

“Employees must submit travel expense reports within 30 days.
Receipts are required for all expenses over \$25.
Late submissions may result in delayed reimbursement.
Managers are responsible for approving expenses.”

Chunk 1: “Employees must submit travel expense reports within 30 days.”

Chunk 2: “Receipts are required for all expenses over \$25.”

Chunk 3: “Late submissions may result in delayed reimbursement.”

Chunk 4: “Managers are responsible for approving expenses.”

If a user asks:

“What happens if I submit a travel expense late?”

Chunking with Overlap

Chunk 1

“Employees must submit travel expense reports within 30 days.
Receipts are required for all expenses over \$25.”

Chunk 2 (overlap included)

“Receipts are required for all expenses over \$25.
Late submissions may result in delayed reimbursement.”

Chunk 3 (overlap included)

“Late submissions may result in delayed reimbursement.
Managers are responsible for approving expenses.”

Question

If you keep characters in a chunk constant and increase chunk overlap. Number of chunks will

- Increase
- Decreases
- No change

What Is RecursiveCharacterTextSplitter?

“RecursiveCharacterTextSplitter” splits text intelligently by trying multiple separators—from larger to smaller—until the chunk size is satisfied. It respects structure first, characters second.

How “Recursive” Works (Core Idea)

It tries to split in this order (default):

1. `\n\n` → paragraphs
2. `\n` → lines
3. `" "` → words
4. Characters → last resort

If a chunk is **still too big**, it **recursively tries the next separator**.

Langchain Chunking Strategies

<https://docs.langchain.com/oss/python/integrations/splitters#text-structure-based>

Quick Question

What is similar?

Car $\leftrightarrow$ Automobile

Cat $\leftrightarrow$ Mat

Embedding Representation

What Happens?

- Each token/word is converted into a **vector representation**/ number.
- These embeddings capture “**semantic meaning**” based on prior training.

Embeddings and Vectors (Math Connection)

Mathematically, an embedding is a vector: $v=[x_1,x_2,x_3,\dots,x_n]\in R^n$

- Each text chunk → one vector in **n-dimensional space**
- Similar meanings → vectors **closer together**
- Different meanings → vectors **far apart**

Embedding Representation - Cntd

What do the numbers mean?

Each number in the vector represents a "feature" of meaning, though often abstract to humans.

Word	Dimension 1 (e.g., Royalty)	Dimension 2 (e.g., Gender)	Dimension 3 (e.g., Age)
King	0.99 (High)	0.99 (Male)	0.7 (Adult)
Queen	0.99 (High)	0.01 (Female)	0.7 (Adult)
Boy	0.01 (Low)	0.99 (Male)	0.1 (Child)

*Simplified conceptual example. Real vectors have 1536+ dimensions.

Embedding models overview: <https://huggingface.co/mteb>

Quick Question

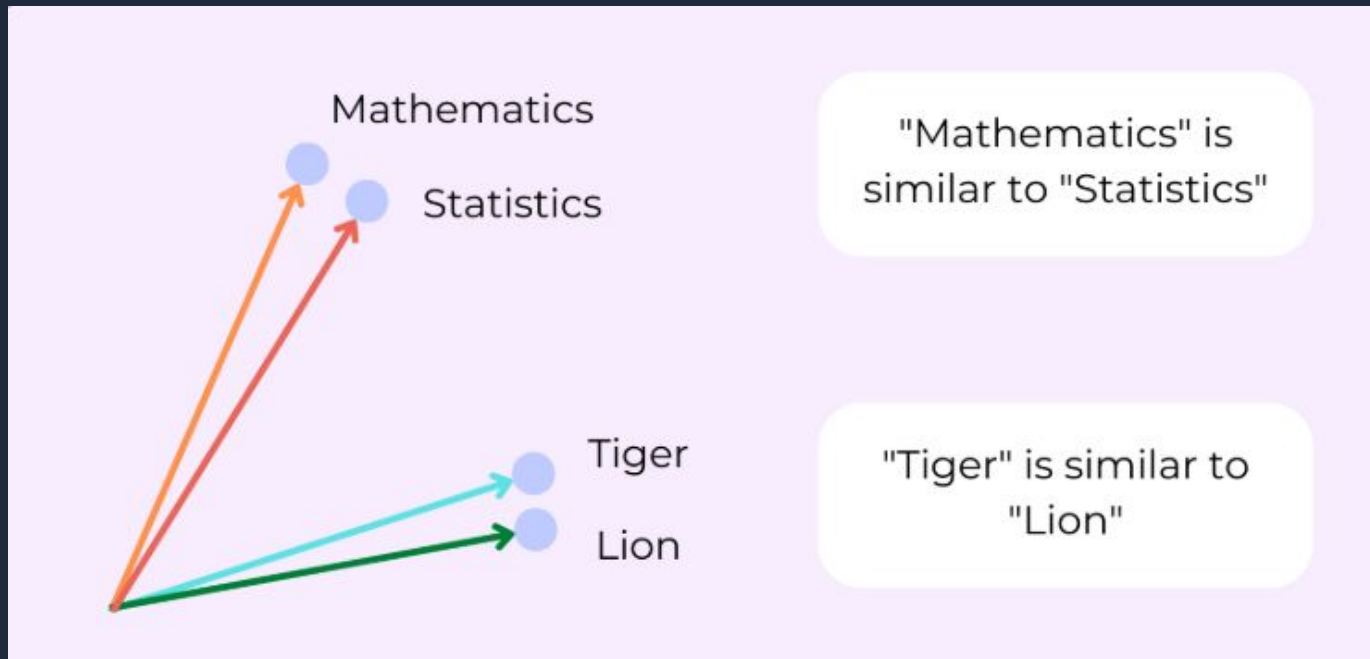
What is Cosine Distance?

Quick Question

What is Euclidean Distance?

Cosine Similarity

Cosine similarity is a way to measure how similar two things are by looking at the **angle** between them in a space, rather than their actual distance.



Imagine Two Arrows

- Think of each thing you want to compare (like two sentences, words, or documents) as an arrow pointing in a specific direction.
- The arrows can be short or long (representing how much information they have), but we are more interested in the **direction** they're pointing, not their length.

What Is a Vector Database?

A **vector database** stores, indexes, and searches **high-dimensional vectors (embeddings)** to enable **semantic similarity search**.

Instead of matching keywords, it finds data whose **vector representations are closest** to a query vector.

Store embeddings → index them → retrieve nearest vectors using distance math

Why Vector Databases Are Needed?

1. Embeddings live in high-dimensional space. Length of the vector (e.g., 768, 1536)
2. Brute-force comparison is slow
3. Vector DBs use Approximate Nearest Neighbor (ANN) algorithms for speed

Popular Vector Databases

Examples (Industry Standard)

- FAISS (local, in-memory)
- Chroma (developer-friendly)
- Pinecone (managed, scalable)
- Weaviate, Milvus (enterprise-grade)

What Is Response Synthesis?

Response synthesis is the step where the LLM:

- Takes **retrieved chunks**
- Combines them
- Produces a **coherent, grounded answer**

Retrieval finds information.

Response synthesis turns information into an answer.

Key concepts - Adding explicit metadata

Add Metadata (So We Can Filter). Update your documents to include metadata:

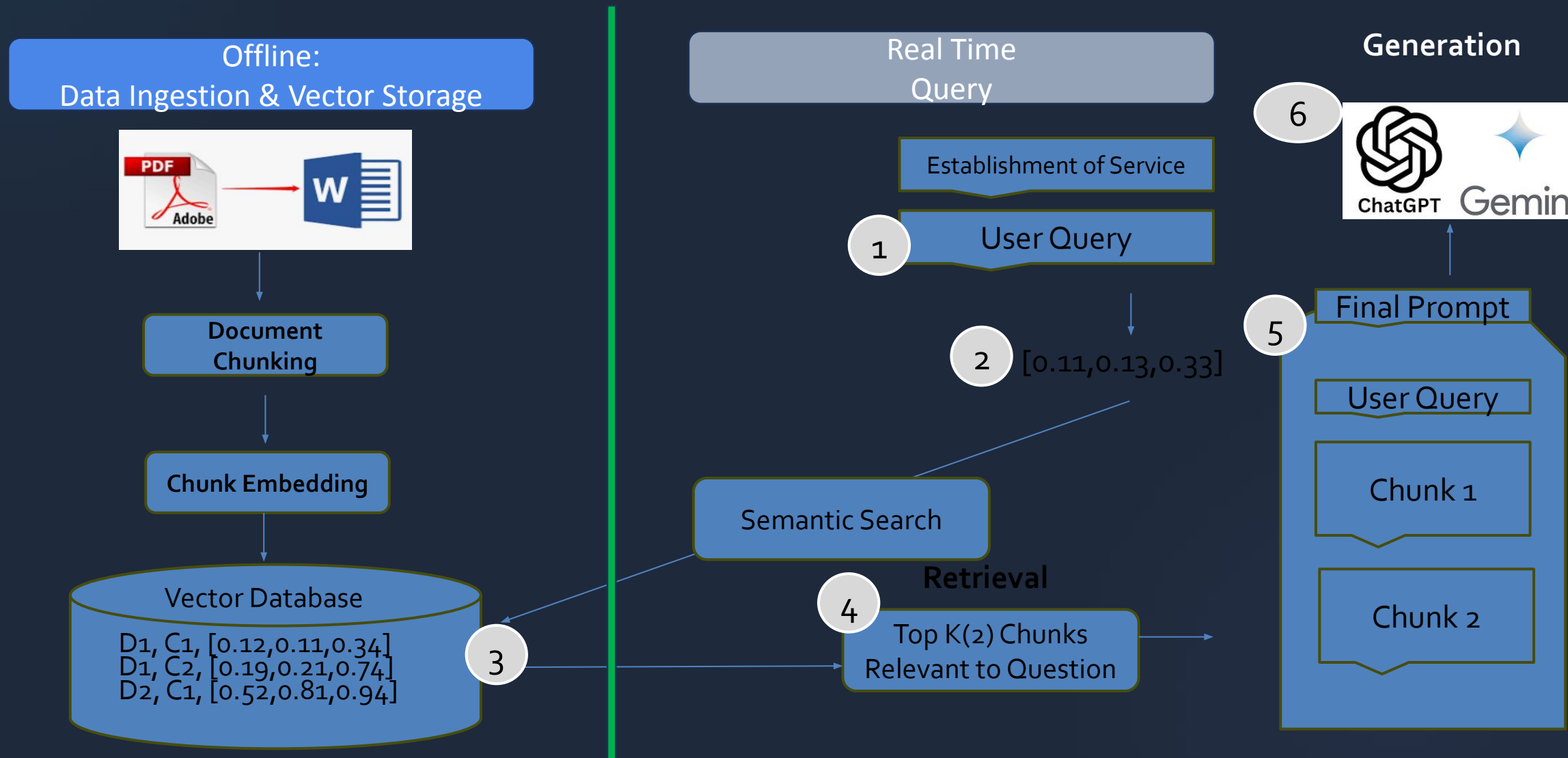
```
from langchain_core.documents import Document
```

```
documents = [  
    Document(  
        page_content="Math is the study of numbers and equations.",  
        metadata={"topic": "math"}  
    ),  
    Document(  
        page_content="Statistics focuses on data analysis and probability.",  
        metadata={"topic": "math"}  
    ),  
    Document(  
        page_content="A cat is a small domesticated animal.",  
        metadata={"topic": "animal"}  
    ),  
    Document(  
        page_content="A mat is an object placed on the floor.",  
        metadata={"topic": "object"}  
    )  
]
```

Key concepts - Metadata Filtering During Retrieval

```
retriever = vectorstore.as_retriever(  
    search_kwargs={  
        "k": 2,  
        "filter": {"topic": "math"}  
    }  
)
```

End to End RAG Pipeline



End to End Hands On RAG Pipeline

The "Naive" RAG Bottleneck

Why Basic RAG Fails?

Simple "Chunk → Embed → Retrieve" pipelines often struggle in production due to:

- The retriever uses the **user's raw query as-is**.
- **Pure Vector Search Might miss Exact Keywords**
- **Top-K Retrieval ≠ Best Ordering**
Retrieved chunks are ordered by **embedding similarity only**.

Technique 1: Query Translation

Fixing the Input

Users write bad queries. We use the LLM to rewrite them *before* retrieval.

- **Multi-Query:** Generating 3-5 variations of the user's question to cast a wider net.
- **HyDE (Hypothetical Document Embeddings):** Asking the LLM to write a fake "ideal answer," then searching for documents that look like that answer.
- **Decomposition:** Breaking a complex multi-hop question into sub-questions.

Technique 2: Hybrid Search

The Best of Both Worlds

Vector search is great for concepts ("Vehicle" matches "Car"), but fails at exact matches (Product IDs, Acronyms).

Sparse (BM25): Traditional keyword search. Excellent for specific terms and exact matches.

Dense (Vectors): Semantic search. Excellent for meaning and intent.

Implementation: Run both searches and combine the results using Reciprocal Rank Fusion (RRF).

Technique 3: Re ranking

The Quality Filter

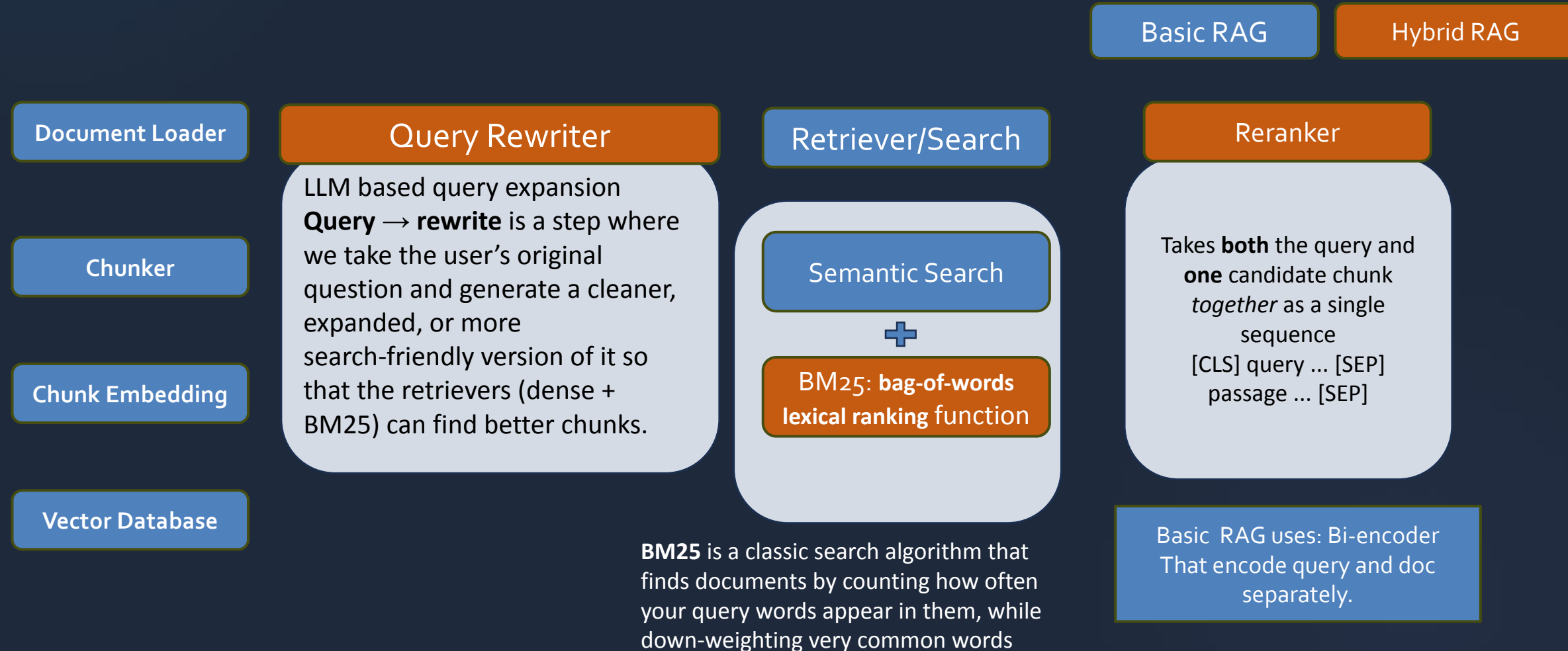
Vector databases are fast but approximate. Re-rankers are slow but precise.

Step 1: Retrieve top 50 documents using fast vector search (Bi-Encoder).

Step 2: Pass those 50 documents + the query into a **Cross-Encoder** (Re-ranker).

Step 3: The Re-ranker assigns a high-fidelity relevance score. Keep only the Top 5 for the LLM.

Advanced RAG: End to end pipeline



Reranker: Reranking Chunks (a re-ranker is a second-pass quality filter)

Basic RAG

Bi-encoder: Encode query and doc separately.

Query: `get_embedding`

Chunk1: `get_embedding`

`Cosine(query_emb, chunk_emb)`

Advanced RAG

A cross-encoder runs **one transformer forward pass over the combined text**.

Takes **both** the query and **one** candidate chunk *together* as a single sequence
[CLS] query ... [SEP] chunk ... [SEP]

Scoring: the model outputs **one relevance score**

- “Retriever = recall, Reranker = precision.”
- “Retriever finds candidates, Reranker decides the order.”
- “We pay a small compute cost to make the LLM’s context cleaner.”

Example

Question

"What is Python used for?"

Document

"Python is a snake found in tropical regions."

"What is Python used for? [SEP] Python is a snake found in tropical regions."

<https://huggingface.co/cross-encoder/ms-marco-MiniLM-L12-v2>

Hands On

Tool Calling Agents

What is Function Calling?

From Text Generators to Action Takers.

| The Paradigm Shift

Before (Standard LLM)

Input: "What is current weather in XYZ location"

Process: Use historic data.

Output: (Often hallucinated/wrong).

After (Function Calling)

Input: "What is current weather in XYZ location"

Process: Detect weather intent.

Output: `call_weather_api(location)`

| Quick check

What is NER (named entity recognition)?

- identification of sentiment
- identification of location, name etc
- Identification of article topic

Hands on

Tasks for Day3:

- **Identify Prompts used for creating SQL agent in langchain. Identify possible issues with that**
- **Identify steps taken by SQL agent behind the scene.**
- **Implement tool calling without Langchain agent**

Image Sources



<https://kanerika.com/wp-content/uploads/2025/10/Evolution-stages-of-AI-Agents-visual-selection.png>

Source: kanerika.com



[https://substackcdn.com/image/fetch/\\$s_!A_Oy!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Fc3177e12-432e-4e41-814f-6febf7a35f68_1360x972.png](https://substackcdn.com/image/fetch/$s_!A_Oy!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Fc3177e12-432e-4e41-814f-6febf7a35f68_1360x972.png)

Source: newsletter.maartengrootendorst.com



https://miro.medium.com/1*5h-GilJPz-yfSdoGDoNcJQ.png

Source: blog.stackademic.com



https://miro.medium.com/1*68h0DAcSLyOlmvgkB-GDzg.png

Source: ai.gopubby.com